

LAB 2: Implementing a Class Diagram - Documentation

The aim of this lab is to implement the design of Seminar 2, that is to build on Lab 1's Vector and Record classes and add the classes Dataset, a StandardizedDataset that returns z-scores, and (optional) a NormalizedDataset that rescales to [0,1]. With also a TestDataset class that exercises all the methods.

We had to define these new classes:

1. Dataset: stores many Record objects and computes statistics.

- Attributes:
 - int dim → input dimension
 - ArrayList<Record> data → aggregation of records
- Constructor:
 - Dataset(int d)
- Getters:
 - int getDim()
 - List<Record> getData()
- Helpers (inputs):
 - Vector meanInput()
 - Vector stdInput()
- Helpers (outputs):
 - double meanOutput()
 - double stdOutput()
- Min–max helpers (for normalization):
 - Vector minInput()
 - Vector maxInput()
 - double minOutput()
 - double maxOutput()
- Other methods:
 - **addRecord(Record r)**: we added checks that print a message and don't add when r is null / r.getInput() is null / the input dimension doesn't match dim.
 - **standardize()**: computes per-feature input mean/std and output mean/std, builds a StandardizedDataset, transforms each record, and returns the new dataset.
 - **normalize()** [optional]: computes per-feature min/max and output min/max, builds a NormalizedDataset, transforms each record, and returns the new dataset.
 - **round5(double)**: rounding to 5 decimals (made protected static so subclasses can reuse it).

2. StandardizedDataset (extends Dataset): represents a dataset where every record is standardized (z-score).

- Attributes:
 - Vector mi, si
 - double mo, so
- Constructor:
 - StandardizedDataset(Dataset d, Vector mi, Vector si, double mo, double so)
- Getters:
 - getMeanIn()
 - getStdIn()
 - getMeanOut()
 - getStdOut()
- Other methods:
 - transform(Record r):
 - Inputs: $(x - mi) / si$

- Output: $(y - m_o) / s_o$
- Rounding: after computing, we round to 5 decimals using `round5` before creating the new Record.
- Zero-variance check: if any `si` component or `so` is 0, we print a one-line message to `stderr` and return the original record unchanged (no exceptions).

3. **NormalizedDataset (extends Dataset) — optional part: min–max scaling to [0,1].**

- Attributes:
 - Vector `minI`, `maxI`;
 - double `minO`, `maxO`.
- Constructor:
 - `NormalizedDataset(Dataset d, Vector minI, Vector maxI, double minO, double maxO)`.
- Getters:
 - `getMinIn()`
 - `getMaxIn()`
 - `getMinOut()`
 - `getMaxOut()`
- Other methods:
 - `transform(Record r)`:
 - Inputs: $(x - \text{minI}) / (\text{maxI} - \text{minI})$ (element-wise)
 - Output: $(y - \text{minO}) / (\text{maxO} - \text{minO})$
 - Rounding: we round the final values to 5 decimals.
 - Zero-range check: if any input range or the output range is 0, we print a message and return the original record.

4. **TestDataset:** tests the implementation from `main()` in the same “works/does not work” style as we did it in Lab 1.

- **For Dataset:**
 - Checks `meanInput()`, `stdInput()`, `meanOutput()`, `stdOutput()` on a small 2-D dataset and compares it to rounded reference values.
- **For StandardizedDataset:**
 - Calls `dataset.standardize()` and verifies the standardized dataset has mean ≈ 0 and std ≈ 1 for inputs and output.
- **For addRecord:**
 - Verifies that null inputs and dimension mismatches are skipped and only valid records are added.
- **For NormalizedDataset (optional):** calls `dataset.normalize()` on a simple dataset and checks the expected three records:
 - $[0.0, 0.0] \rightarrow 0.0$, $[0.5, 0.5] \rightarrow 0.5$, $[1.0, 1.0] \rightarrow 1.0$
- **Extra zero-variance/zero-range checks:**
 - Creates datasets where a feature or the output is constant; `transform` prints an error and leaves records unchanged, which the test asserts.

Alternatives solutions we discussed were for instance, for error handling, we considered either throwing exceptions or simply printing a short message and skipping/returning the original record; we chose the latter to keep tests and console output simpler

Finally, for rounding, we debated rounding intermediate statistics (means/stds/mins/maxes) versus keeping full precision and rounding only final returned values; we chose to round only at the end (and when printing) since rounding mins/maxes early sometimes made the ranges fail and give errors and cause string-comparison mismatches.

We applied several object-oriented programming concepts throughout this Lab. Inheritance is used by `StandardizedDataset` and `NormalizedDataset`, which extend `Dataset` and inherit methods like `addRecord()`, `getDim()`, and `toString()`. We use aggregation/composition since

Dataset aggregates Record. Encapsulation is used by keeping internal arrays private and exposing state only through getter methods

The helper method `round5` is shared as a protected static method to be able to use it in other classes (reuse). Finally, we respect Liskov substitution because both specialized datasets can be used anywhere a Dataset is expected without breaking behavior.

Overall, we hit a few difficulties but we solved and fixed everything. For example, we analyzed the zero-variance/zero-range cases that could cause $1 / 0 = \text{infinity}$, so we added tiny exception checks in `transform()` that print a message and return the original record if this happens.

In addition, `addRecord` originally accepted anything, so we added short null and dimension checks to avoid errors with the dataset. Also we had trouble with rounding placement, rounding mins/maxes (and means) too early made `normalize()` “fail,” so we decided to compute with full precision and round only the final transformed values with our helper method `round5`. We also made `round5` protected static to reuse it from subclasses

We also had difficulties with string comparisons in tests because of `toString()` formatting, so we either matched the exact format and ensured 5 decimal rounding for stable outputs.

With these fixes, the tests show correct means/stds, successful standardization (means ≈ 0 , stds ≈ 1), correct normalization to `[0,1]`, and handling of invalid adds and zero-range cases. Everything now passes with 0 errors.